Analysis of GALAH dataset using Convolutional Neural Networks (CNN)

Marco Maltoni

28/04/2026

1 The analysis

The presented dataset from the GALAH experiment contains 8914 spectra from the observation of different stars, each containing 16384 data points (figure 1). The targets to predict were presented as 8914 arrays containing various values in different ranges, from which only three elements, associated with the labels T_{eff}, $log\ g$ and $\frac{Fe}{h}$, were selected. The task was interpreted as a regression problem: at first, due to initial convergence problems of the loss function, the targets were normalized, shifting their mean value to zero and dividing by the standard deviation.

$$target_{norm} = \frac{target - mean_{target}}{std_{target}}$$

After this, the standard procedure was followed: the dataset was divided into training (0.6), evaluation (0.2) and testing (0.2) sets and loaded on the *"DataLoader"*, choosing the *"T4 GPU"* as the device. Then the model was implemented: to quickly reduce the dataset size in the first layer to gain a decrease of compiling time, a stride was applied, and padding was avoided for the first two convolutional layers. $ReLU$ was chosen as the activation function for the whole process, *"Mean Square Error"* as the loss function, and *"Adam"* as the optimizer. After three convolutional layers comes an FCNN of three layers ending with a 3-neuron output layer without an activation function, as the targets are three (figure 2). The results of the training look promising, as the loss functions (figure 3) both from the training and evaluation sets decrease with the number of epochs and converge. The fact that the functions present a similar shape is a hint of the absence of overfitting. The results of the testing show the model is efficient too, as the points obtained by plotting the predictions with respect to the real data lie around the identity line with respect to the axes (figure 4). During the exercise, a few challenging aspects have been encountered: in particular, how to find a combination of kernel dimensions for the convolutional and pooling layers that fitted the initial dimension of data and how to reduce the training time. About the possible improvements of the model, changing the

hyperparameters (batch size, learning rate, etc.) or increasing the number of layers could be solutions to increase the efficiency and avoid possible biases.

2 Plots

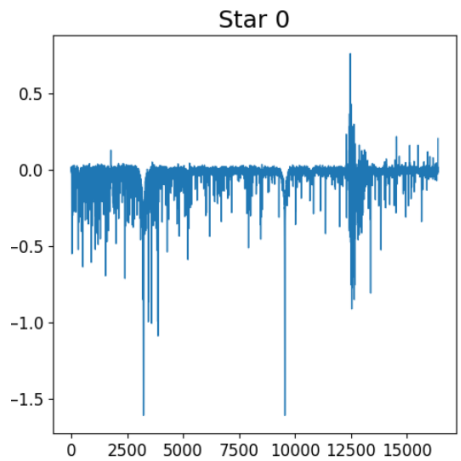

Figure 1: Example of a normalized set of data from a spectrum.

```
class CNN(nn.Module):
  def __init__(self):
    super(CNN, self).__init__()
    self.conv = nn.Sequential(
        nn.Conv1d(1,32,kernel_size = 15, stride = 2, padding = 0), #Output are 32 tensors of 8185 elements
        nn.ReLU(),
        nn.MaxPool1d(kernel_size = 5), #reduction of dimension 4, output are 32 tensors of 1637 elements
        nn.Conv1d(32,32,kernel_size = 13, padding = 0), #Output are 32 tensors of 1625 elements
        nn.ReLU(),
        nn.MaxPool1d(kernel_size = 5), #Output are 32 tensors of 325 elements
        nn.Conv1d(32,16,kernel_size = 15,padding='same'), #Output are 16 tensors of 325 elements
        nn.ReLU(),
        nn.MaxPool1d(kernel_size = 5), #Output are 16 tensors of 65 elements
        nn.Flatten(),
        nn.Linear(16*65,256), #Start of FCNN
        nn.ReLU(),
        nn.Linear(256,3)
    )

  def forward(self,x):
    return self.conv(x)
```

Figure 2: Scheme of implemented model with dimension of the output for each layer.

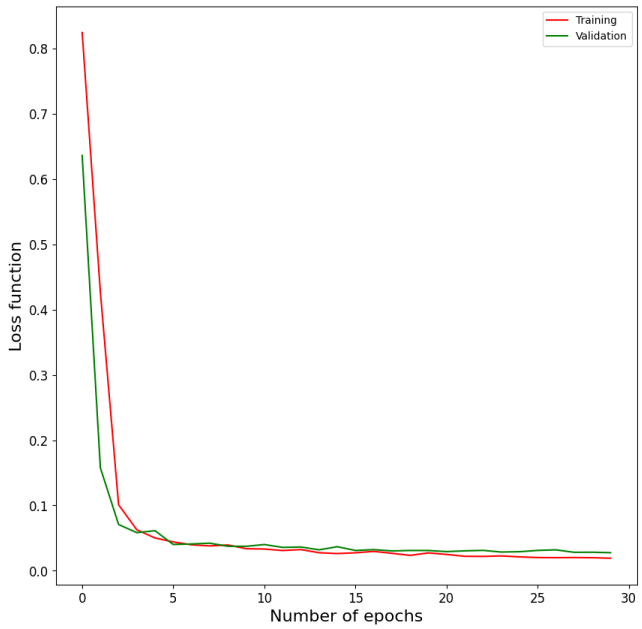

Figure 3: Loss values for the training and evaluation datasets. The plots show an expected behavior as they decrease during the training (evaluation) procedure and in the end they converge to a low value.

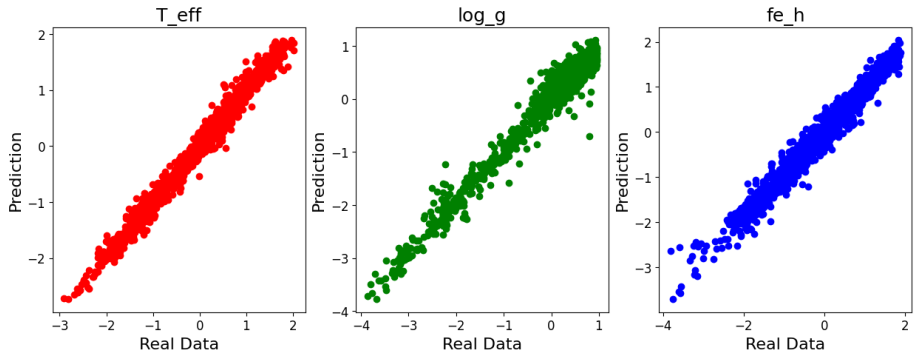

Figure 4: Plots of the predicted values from the testing procedure with respect to the related targets (normalized). The points appear around the identity line: the more the values on the two axes are close, the more the performance of the trained model is efficient.